Análise qualitativa da revisão dos prompts de Produto

Comparação antes/depois do pacote *esteira-anosfinais-formato-produto*
e avaliação da granularidade da esteira (9 agentes *versus* 5)

Análise técnica — repositório `gerador-missoes` (Unooria)

18 de setembro de 2026

Resumo

Este relatório compara, arquivo a arquivo, os prompts da área de Produto vigentes no repositório `gerador-missoes` (*HEAD* `914f05b`) com o pacote revisado entregue pelo *Product Owner* (pasta `esteira-anosfinais-formato-produto`, 18/09/2026). Foram analisadas 13 mudanças distintas distribuídas em 11 arquivos. O resultado não é homogêneo: seis mudanças são melhorias substantivas e tecnicamente defensáveis — uma delas corrige um erro de desenho de avaliação que a literatura de tolerância a falhas prevê desde 1986 —, duas são regressões graves que quebram contratos determinísticos a jusante, e duas exigem alteração de código para funcionar. A recomendação é **integração seletiva**, não adoção em bloco nem rejeição em bloco. O relatório inclui ainda uma avaliação da granularidade da esteira, concluindo que a decomposição atual em nove agentes é superior a uma consolidação em cinco — mas por um motivo diferente do senso comum de que "mais especialistas é melhor".

Sumário

1 Sumário executivo **3**
- 1.1 Veredito . . . 3
- 1.2 Os três achados que importam . . . 3
- 1.3 Resposta direta às duas perguntas feitas . . . 3

2 Método e escopo **4**
- 2.1 O que foi comparado . . . 4
- 2.2 Procedimento . . . 4
- 2.3 Limites desta análise . . . 4

3 Contexto: o que a esteira faz e onde estão os pontos de controle **5**

4 Análise mudança a mudança **6**
- 4.1 Visão geral . . . 6
- 4.2 M1 — Veredito binário para ternário . . . 6
- 4.3 M2 — O envelope `aprovado_com_ressalvas` não tem contraparte no código . . . 7
- 4.4 M3 — Regra de parada do *loop* P1↔P1b . . . 8
- 4.5 M4 — Remoção dos critérios 3.1–3.3 do P3 . . . 8
- 4.6 M5 — Operacionalização do critério 1.6 (reflexão final) . . . 9
- 4.7 M6 — "Teste de cenário real" no critério 6.3 . . . 10
- 4.8 M7 — Poda de ancoragem nos critérios 6.1, 6.4 e 6.5 . . . 10
- 4.9 M8 — "aluno" para "estudante" . . . 11
- 4.10 M9 — Material *Entre Redes* como fonte primária de BNCC . . . 11
- 4.11 M10 — O novo agente P5b . . . 12
- 4.12 M11 — `formato-spec.md`: a regressão grave . . . 12

4.13 M12 — `ambientes.md`: reversão e erro factual 14
4.14 M13 — Remoção do *frontmatter* YAML 14

5 Efeito líquido sobre a construção da missão **15**
5.1 Quantificação 15
5.2 Matriz de decisão 15
5.3 Como o pacote ajuda a construção 16
5.4 Como o pacote dificulta 16

6 Granularidade da esteira: nove agentes ou cinco? **17**
6.1 A pergunta 17
6.2 O argumento a favor da decomposição 17
6.3 O argumento contrário, e por que ele não se aplica aqui 18
6.4 O critério de decisão 18
6.5 Conclusão e recomendação 18

7 Recomendações **20**

8 Conclusão **20**

A Evidências verificadas **22**

1 Sumário executivo

1.1 Veredito

O *Product Owner* entregou um pacote de qualidade técnica **acima** do que a desconfiança inicial sugeria. As mudanças conceituais — veredito ternário, desduplicação de critérios, regra de parada de *loop*, operacionalização de rubrica — são corretas, bem argumentadas e sustentadas por literatura consolidada. O problema não está no julgamento pedagógico; está em **higiene de integração**: o pacote foi editado sobre um *snapshot* desatualizado do repositório e desfaz, sem intenção aparente, dois *commits* de sincronização de contratos de plataforma.

Tabela 1: Distribuição das 13 mudanças identificadas

Classe	Qtd.	Ação recomendada
Melhoria substantiva	5	Integrar
Melhoria com ajuste necessário	3	Integrar após correção pontual
Regressão grave	2	**Descartar** (reverter para o estado atual)
Regressão leve	2	Descartar ou revisar caso a caso
Neutra	1	Indiferente

1.2 Os três achados que importam

1. **A revisão do `formato-spec.md` é uma reversão, não uma revisão** (Seção 4.12). O arquivo revisado é bit-a-bit equivalente à versão do *commit* `4dcdb18` (16/09), **anterior** ao `262f160` que sincronizou os contratos da plataforma. Ele remove 73 linhas, entre elas a ficha do manifesto — cujos dez campos são lidos por um validador determinístico (`tools/rastro.py`) que emite severidade *bloqueio* para cada campo ausente. Adotar este arquivo faz **toda** missão nova travar no gate de manifesto da etapa de Dev.

2. **O novo valor de envelope `aprovado_com_ressalvas` não existe no código** (Seção 4.3). A função `agents/callbacks.py::inferir_status` reconhece `apto_com_ressalvas`, `ressalvas`, `amarelo` e o emoji amarelo, mas não `aprovado_com_ressalvas`. O envelope cai no *fallback* textual, cujo teste de bloqueio é avaliado **antes** do teste de aprovação — o que abre um caminho concreto de falso bloqueio. Correção: duas linhas.

3. **A desduplicação dos critérios 3.1–3.3 é o melhor achado do pacote** (Seção 4.5). O PO identificou que três critérios obrigatórios do agente P3 eram testes idênticos aos critérios 1.3, 1.4 e 7.2 do agente P1. A verificação no repositório confirma a equivalência literal. Redundância cujos avaliadores falham de forma correlacionada não adiciona cobertura — é o resultado clássico de Knight e Leveson [7] sobre programação multiversão, e vale com ainda mais força quando os dois "avaliadores independentes" são *prompts* diferentes do mesmo modelo.

1.3 Resposta direta às duas perguntas feitas

Os prompts melhoraram ou pioraram?

Os **prompts de agente** (`p1`–`p5` e o novo `p5b`) melhoraram. Os **arquivos de referência** que eles incluem pioraram, e é por eles que a esteira quebra.

Ajudam ou dificultam a construção da missão?

As mudanças de veredito e de *loop* ajudam a construção e, principalmente, a auditoria. As mudanças nos arquivos de referência dificultam de forma determinística: o contrato que Design e Dev consomem fica incompleto.

2 Método e escopo

2.1 O que foi comparado

- **Linha de base:** `prompts/produto/` e `prompts/references/` no *HEAD* `914f05b` da *branch* `main`, com a árvore de trabalho limpa para esses caminhos.
- **Revisão:** os 6 arquivos de *prompt* e os 10 arquivos de referência da pasta `~/Downloads/esteira-anosfinais` (carimbo de tempo 18/09/2026, 18h43).

2.2 Procedimento

A análise combinou quatro instrumentos, nesta ordem:

1. **Diferencial textual completo** (`diff -u`) de cada par de arquivos, com quantificação de linhas removidas e adicionadas.
2. **Arqueologia de versão**: comparação de cada arquivo revisado contra as versões históricas do *git*, para distinguir *edição* de *reversão*. Este passo isolou os achados M11 e M12.
3. **Verificação contra o código executável**: leitura dos consumidores de cada contrato alterado (`agents/callbacks.py`, `agents/checkers.py`, `agents/produto/__init__.py`, `tools/rastro.py`, `tools/bncc.py`, `pipeline/session_store.py`, `config.py`), para determinar se a mudança de *prompt* tem contraparte no código.
4. **Teste empírico** onde a leitura de código deixava dúvida. Duas hipóteses foram testadas executando o próprio validador do repositório; uma foi confirmada e **outra foi refutada** (Seção 4.10) — registrada aqui porque a refutação muda o veredito daquela mudança.
5. **Medição de densidade de instrução** por agente, com resolução das diretivas `<!- include: ... ->`, para fundamentar a discussão de granularidade (Seção 6).

2.3 Limites desta análise

Esta é uma análise **estática e de contrato**. Ela não executa a esteira ponta a ponta com os prompts revisados, e portanto não mede efeito sobre a qualidade final das missões geradas — o que exigiria um experimento com n missões, dois braços e avaliação cega por especialistas humanos (proposto na Seção 7, item R7). As afirmações sobre bloqueio determinístico são verificáveis por leitura de código e foram confirmadas; as afirmações sobre qualidade de julgamento do modelo são argumentos fundamentados, não medições.

3 Contexto: o que a esteira faz e onde estão os pontos de controle

A etapa de Produto é um `SequentialAgent` com nove agentes e dois *loops* de correção:

1. **P0a** — núcleo narrativo (seções 0, 1, 3 e 4 do *one-pager*);
2. **P0mg** — micro-gate de *spoiler*, *gap* de informação e escolha significativa (*loop* com P0a, teto de 2 rodadas);
3. **P0b** — seções de apoio (completa as 18 seções do *one-pager* canônico);
4. **P1** — gate estrutural (23 critérios);
5. **P1b** — revisor do *one-pager* (*loop* com P1, teto de 3 rodadas por `GATE_ESTRUTURAL_MAX_RODADAS`);
6. **P2** — avaliação pedagógica (20 critérios) — em paralelo com P3;
7. **P3** — experiência do estudante (11 critérios hoje, 8 na revisão);
8. **P4** — gate final de produto;
9. **P5** — *spec* da missão (12 blocos), o contrato contra o qual Design e Dev constroem.

Entre cada agente e o seguinte há um `EnvelopeGateAgent`, que lê o bloco JSON final do relatório, normaliza o veredito e decide se o *pipeline* segue, repete ou para. Com `PRODUTO_MODO_ESTRITO=true` (o padrão em `.env`), um gate bloqueado impede a execução de Design e Dev. **É esse envelope que o pacote revisado altera** — e é por isso que uma mudança aparentemente redacional tem consequência operacional.

A jusante, na etapa de Dev, há três gates **determinísticos** (não são modelos de linguagem, são funções Python): `lacunas`, `manifesto` e `telemetria`. O gate de manifesto lê a ficha do bloco 0 da *spec* produzida pelo P5. Esse é o ponto exato onde a regressão M11 se manifesta.

4 Análise mudança a mudança

4.1 Visão geral

Tabela 2: As 13 mudanças identificadas, com classificação

#	Mudança	Onde	Veredito
M1	Veredito binário → ternário	p1, p2, p3, p4	**Melhoria** (com salvaguarda)
M2	Envelope `aprovado_com_ressalvas`	p1, p2, p3, p4	**Melhoria bloqueada por código**
M3	Regra de parada do *loop* P1↔P1b	p1	**Melhoria** (com *off-by-one*)
M4	Remoção dos critérios 3.1–3.3 do P3	p3, criterios-experiencia	**Melhoria forte**
M5	Operacionalização do critério 1.6	criterios-gate-estrutural	**Melhoria forte**
M6	"Teste de cenário real" no critério 6.3	criterios-experiencia	**Melhoria parcial**
M7	Poda de ancoragem em 6.1, 6.4, 6.5	criterios-experiencia	**Regressão leve**
M8	"aluno" → "estudante"	vários	Neutra/positiva
M9	Material *Entre Redes* como fonte primária BNCC	p2, p5	**Risco condicional**
M10	Novo agente P5b (*one-pager* pedagógico)	arquivo novo	**Melhoria** (falta *wiring*)
M11	`formato-spec.md` revertido (–73 linhas)	references	**Regressão grave**
M12	`ambientes.md` revertido (–9 linhas)	references	**Regressão**
M13	Remoção do *frontmatter* YAML	4 references	**Regressão leve**

4.2 M1 — Veredito binário para ternário

O que mudou. Os quatro agentes avaliadores passam de um veredito de duas categorias (APROVADO / BLOQUEADO) para três (APROVADO / APROVADO COM RESSALVAS / BLOQUEADO). O texto anterior do P1 era explícito na direção oposta: *"Aprovado com ressalvas não existe"*.

Por que é uma melhoria. O sistema anterior não eliminava o estado intermediário — ele o **comprimia**. O P2 anterior instruía, literalmente: *"Use `aprovado` para 'pedagogicamente pronta' e 'pronta com ressalvas'"*, e o P3 fazia o mesmo. Ou seja, a distinção existia no relatório em prosa e era destruída no campo que o código lê. Um consumidor a jusante — outro agente ou uma pessoa olhando o painel — via "aprovado" sem forma de saber se havia risco aceito.

Isso é perda de informação por desenho. Na literatura psicométrica, escalas de 2 e 3 pontos apresentam desempenho sistematicamente inferior em confiabilidade, validade e poder discriminante quando comparadas a escalas com mais categorias; Preston e Colman [12] mediram os índices subindo de forma significativa até cerca de sete categorias. A passagem de 2 para 3 é um ganho de discriminação com custo baixo de concordância.

O risco que isso cria. A categoria intermediária é também a categoria de fuga. A literatura clássica sobre erros de avaliador descreve dois vieses convergentes aqui: *leniency* (tendência sistemática a pontuar acima do devido) e *central tendency* (evitar os extremos da escala), ambos

catalogados por Saal, Downey e Lahey [13]. O fenômeno é documentado especificamente em avaliadores baseados em modelos de linguagem: Zheng et al. [17] caracterizam vieses de posição, verbosidade e autopreferência, e trabalhos posteriores registram *leniency* como viés recorrente — consequência particularmente séria em rubricas dominadas por critérios negativos, exatamente o caso desta esteira, onde o sinal que a rubrica existe para capturar é a **taxa de erro**.

Sem salvaguarda, o resultado esperado é deriva: o **AMARELO** vira o veredito confortável, e a proporção de **VERDE** puro cai sem que ninguém decida isso.

Salvaguarda recomendada. O pacote já traz metade da proteção: o campo `ressalvas: <n>` no envelope, que torna a contagem observável. Falta a outra metade — uma política explícita. Sugestão mínima:

1. registrar `ressalvas` por missão e monitorar a série temporal (se a média subir sem queda correspondente em `bloqueios_obrigatorios`, é deriva, não melhoria de qualidade);
2. definir um teto por missão acima do qual **AMARELO** escala para decisão humana (`precisa_humano: true`), em vez de seguir;
3. manter a exigência — que o pacote preserva corretamente — de que cada ressalva sobrevivente nomeie os agentes a jusante e demonstre que nenhum precisa daquilo.

4.3 M2 — O envelope `aprovado_com_ressalvas` não tem contraparte no código

O problema. O pacote introduz o valor `"status": "aprovado_com_ressalvas"` nos envelopes de P1, P2, P3 e P4. A função que normaliza esse campo é:

Listing 1: `agents/callbacks.py`, linhas 55–73 (estado atual)

```
def inferir_status(texto, envelope=None):
    if envelope:
        status = str(envelope.get("status") or envelope.get("veredito") or "").strip().lower()
        if status in ("aprovado", "concluido", "concluído", "liberada", "apto", "passa", "verde",
              "<verde>"):
            return STATUS_APROVADO
        if status in ("bloqueado", "bloqueada", "devolvido", "nao_apto", ...):
            return STATUS_BLOQUEADO
        if status in ("apto_com_ressalvas", "ressalvas", "amarelo", "<amarelo>"):
            return STATUS_APROVADO
        ...
    cabecalho = (texto or "")[:1500].upper()
    if any(tok in cabecalho for tok in _BLOQUEADO_TOKENS): # testado PRIMEIRO
        return STATUS_BLOQUEADO
    if any(tok in cabecalho for tok in _APROVADO_TOKENS):
        return STATUS_APROVADO
    return STATUS_INDEFINIDO
```

O vocabulário aceito contempla `apto_com_ressalvas` e `amarelo` — mas **não** `aprovado_com_ressalvas`. O envelope, que é o caminho robusto e estruturado, deixa de decidir, e a decisão cai no *fallback* heurístico sobre os primeiros 1.500 caracteres do texto.

Por que o *fallback* é perigoso aqui. Os tokens de bloqueio são testados antes dos de aprovação, e a linha de cabeçalho que o próprio *prompt* manda emitir é:

```
**Veredito: <verde> APROVADO | <amarelo> APROVADO COM RESSALVAS | <vermelho> BLOQUEADO** ...
```

Se o modelo reproduzir a linha do gabarito em vez de escolher uma opção — comportamento comum quando o gabarito lista alternativas na mesma linha —, o cabeçalho contém o emoji vermelho e `BLOQUEADO`, e a missão é classificada como bloqueada. Com `PRODUTO_MODO_ESTRITO=true`, isso mata a esteira: Design e Dev são pulados e o PRD não sai.

Esse risco já existia no formato binário, mas ali o caminho do envelope funcionava e absorvia o erro. Ao introduzir um valor não reconhecido, a revisão **remove a rede de proteção** e deixa apenas a heurística. É uma violação de contrato no sentido de Meyer [9]: o produtor passou a emitir um valor fora do domínio que o consumidor declara aceitar, e a falha resultante não é local — é silenciosa e a jusante.

Impacto secundário. `pipeline/session_store.py::formatar_status_label` também não conhece o valor: a interface exibiria "(sem cor) Aprovado_com_ressalvas" em vez do rótulo amarelo correto.

Correção. Trivial, duas linhas — acrescentar `aprovado_com_ressalvas` às duas listas que já contêm `apto_com_ressalvas`. A alternativa (padronizar os prompts em `apto_com_ressalvas`, valor que o código já aceita) evita tocar no código, mas desalinha o vocabulário dos prompts entre si. **Recomenda-se corrigir o código**, que é a fonte mais estável.

4.4 M3 — Regra de parada do *loop* P1↔P1b

O que mudou. O P1 revisado ganha uma seção nova: teto de três rodadas de correção; ao avaliar a rodada seguinte com a missão ainda bloqueada, o agente deixa de emitir o bloco "Complemento — pronto pra colar" e passa a emitir "Limite de rodadas atingido — decisão humana necessária", marcando `precisa_humano: true`. A justificativa escrita no próprio *prompt* é precisa: o P1b nunca decide questões de produto (é regra dele), logo, se o bloqueio for uma decisão pendente, mais uma rodada não pode resolvê-lo por construção.

Por que é uma melhoria. Terminação e verificação são categorias inteiras da taxonomia de falhas de sistemas multiagente de Cemri et al. [1], construída sobre mais de 1.600 traços anotados em sete *frameworks* e validada com concordância entre anotadores de $\kappa = 0{,}88$. Uma das famílias de falha é justamente o sistema insistir em ciclos de correção que não podem convergir, porque a informação necessária não está disponível para nenhum dos agentes. Tornar essa condição explícita e escalá-la para um humano é o tratamento correto.

A regra também distingue as duas causas do bloqueio persistente — falta de mecanismo (o P1b poderia ter resolvido e não resolveu) *versus* decisão de produto em aberto (nenhuma rodada resolve). Essa distinção é o que transforma "o *loop* estourou" em informação acionável.

O defeito. A regra é escrita para disparar "ao avaliar a **rodada 4** (ou além)". O *loop* é um `LoopAgent` com `max_iterations = GATE_ESTRUTURAL_MAX_RODADAS`, cujo padrão em `.env` e em `config.py` é **3**. A rodada 4 nunca ocorre: o *loop* encerra na terceira. A instrução, como escrita, é letra morta, e o `precisa_humano: true` que ela pretendia garantir nunca é emitido por esse caminho.

Correção. Trocar o gatilho para a rodada final — "na 3.ª rodada, se a missão continuar bloqueada" — ou, melhor, parametrizar o texto do *prompt* pelo mesmo valor de configuração, já que `load_prompt` suporta substituição de variáveis estáticas via *kwargs*. Isso elimina a possibilidade de o *prompt* e o *runner* divergirem de novo quando alguém mudar a variável de ambiente.

4.5 M4 — Remoção dos critérios 3.1–3.3 do P3

O que mudou. O agente P3 perde três critérios obrigatórios — 3.1 (ação com significado), 3.2 (o resultado reflete a ação) e 3.3 (sem julgamento do estudante como pessoa) — e passa de 11 para 8 critérios (5 obrigatórios + 1 condicional + 2 desejáveis). A justificativa registrada no arquivo de referência é que eram testes idênticos aos critérios 1.3, 1.4 e 7.2 do gate estrutural.

Verificação. A alegação foi conferida contra `prompts/references/criterios-gate-estrutural.md` e é **literalmente verdadeira**:

Tabela 3: Equivalência entre os critérios removidos do P3 e os do P1

P3	Teste prático (removido)	P1	Teste prático (mantido)
3.1	"Se tirar a ação do estudante, a missão continua chegando no mesmo lugar? Se sim → falha"	1.3	"Se tirar a decisão, a missão continua funcionando igual? (se sim, falha)"
3.2	"O estudante percebe a relação entre o que fez e o que aconteceu?"	1.4	"O aluno vê o impacto da sua escolha? Caminhos diferentes geram resultados diferentes?"
3.3	"Alguma métrica nascida na simulação vaza pra avaliação formal fora dela?"	7.2	"Alguma métrica da simulação vaza para avaliação formal (boletim, conceito)?"

Por que é uma melhoria forte. A intuição contrária — "duas verificações são melhores que uma" — pressupõe independência de falhas. Knight e Leveson [7] testaram exatamente essa premissa em 27 implementações independentes do mesmo requisito, submetidas a um milhão de testes, e encontraram correlação estatisticamente significativa entre as falhas: programas escritos separadamente erravam nos *mesmos* casos, com frequência muito acima do previsto pela hipótese de independência. A conclusão é que redundância só compra confiabilidade na medida em que as falhas são descorrelacionadas.

No caso desta esteira, a premissa de independência é ainda mais frágil: P1 e P3 não são implementações independentes — são *prompts* distintos executados pelo **mesmo modelo**, sobre o **mesmo artefato**, com o **mesmo texto de teste**. A probabilidade condicional de o P3 detectar uma violação de 3.1 que o P1 deixou passar em 1.3 é baixa por construção. O que a duplicata efetivamente produzia não era cobertura extra, e sim três itens ocupando espaço de atenção no P3.

O ganho secundário é o principal. O P3 existe, segundo o texto do próprio *prompt*, porque *"a rubrica original errava sistematicamente aqui: os critérios de faixa etária competiam por atenção com ∼30 critérios de outra natureza"*. Retirar três itens redundantes devolve capacidade de atenção à lente que só o P3 tem — carga cognitiva, adequação etária, simulação do estudante real. O mecanismo é o mesmo que motivou a criação do agente, e é consistente com a evidência de que a aderência a instruções degrada de forma não linear com a densidade: no *benchmark* IFScale, de Jaroslawicz et al. [4], os melhores modelos de fronteira atingem apenas 68% de acurácia na densidade máxima testada, com viés sistemático em favor das instruções iniciais.

Ressalva. A remoção é segura **apenas enquanto** P1 e P3 permanecerem ambos no *pipeline* e na ordem atual. Se algum dia o P1 for reorganizado, esses três critérios saem da esteira sem aviso. Recomenda-se registrar a dependência explicitamente no cabeçalho de `criterios-gate-estrutural.md` — o pacote já a documenta do lado do P3, falta o lado do P1.

4.6 M5 — Operacionalização do critério 1.6 (reflexão final)

O que mudou. O critério 1.6 ganha uma enumeração de três armadilhas (campo aberto genérico, quiz de múltipla escolha, e — nova — provocação que contém a resposta na própria pergunta) e, sobretudo, um **teste de falseabilidade**: *"um estudante que NÃO fez a missão consegue responder corretamente só pela pergunta? Se sim, a provocação não testa aplicação — testa leitura."*

Por que é uma melhoria forte. Esta é a mudança mais bem construída do pacote do ponto de vista de instrumento de avaliação. A revisão sistemática de Jonsson e Svingby [5], sobre 75 estudos empíricos de rubricas, conclui que a confiabilidade de pontuação melhora com rubricas **analíticas e específicas ao tópico**, complementadas por exemplos concretos — que é precisamente a forma da mudança: sai um critério declarativo, entra um procedimento com contraexemplos nomeados e um teste de decisão.

O teste proposto tem uma propriedade rara em rubrica: é **falseável sem consultar a intenção do autor**. O avaliador não precisa julgar se a provocação "é boa"; precisa executar um experimento mental com resultado binário. Isso reduz a variância entre execuções, que é a fonte principal de ruído quando o avaliador é um modelo de linguagem.

4.7 M6 — "Teste de cenário real" no critério 6.3

O que mudou. O critério de complexidade da decisão ganha uma terceira dimensão: a calibração deve ser feita para o **cenário de piso** (escola pública brasileira, estudante possivelmente sem familiaridade com a plataforma simulada, sem dispositivo próprio, sem compreensão de instruções implícitas) e não para o cenário ideal.

Avaliação. A intenção é correta e pedagogicamente relevante: calibrar para o cenário ideal é uma forma silenciosa de excluir a maior parte do público-alvo real, e a preocupação conversa diretamente com a teoria da carga cognitiva — a carga extrínseca imposta pela interface e pelas instruções compete pelos mesmos recursos que a tarefa de aprendizagem [14], e essa competição é maior justamente para quem tem menos familiaridade prévia.

O problema é de **operacionalização**, e o contraste com M5 é instrutivo. Enquanto o 1.6 revisado entrega um teste binário executável, o 6.3 revisado entrega uma pergunta aberta ("foi calibrada para o ideal ou para o piso?") cujo resultado depende inteiramente do julgamento não ancorado do avaliador. Em rubrica, isso tende a produzir ruído: o mesmo artefato recebe vereditos diferentes em execuções diferentes, que é o modo de falha que a rubrica analítica existe para evitar [5].

Sugestão. Converter em uma *checklist* de três perguntas verificáveis, no mesmo formato do 1.6: (a) a missão declara o que acontece quando o estudante não conhece a plataforma simulada? (b) existe caminho de conclusão com dispositivo compartilhado? (c) alguma instrução é implícita, isto é, depende de um gesto que a missão nunca demonstra? Cada uma tem resposta objetiva no *one-pager*.

4.8 M7 — Poda de ancoragem nos critérios 6.1, 6.4 e 6.5

O que mudou. Ao reescrever `criterios-experiencia.md`, o pacote encurtou o texto de vários testes práticos, removendo: a fundamentação cognitiva do critério 6.1 (a referência a Piaget e o argumento de que a decodificação de texto compete com a atenção disponível para decisão em crianças de 6–7 anos); o exemplo concreto do critério 6.4 ("usar WhatsApp para uma atividade que a criança faria no Instagram"); e a explicitação do 6.5 ("passa em todos os obrigatórios individualmente mas resolve rápido demais").

Por que conta como regressão. São exatamente os elementos que a literatura identifica como responsáveis pela confiabilidade de uma rubrica: exemplos, contraexemplos e justificativa do limiar [5]. A economia de texto é pequena — o arquivo inteiro encolheu cerca de 300 caracteres — e o que se perde é a âncora que faz dois avaliadores convergirem. Vale notar que a fundamentação removida do 6.1 não é decorativa: ela é a razão pela qual a exigência de narração em áudio existe para 1.º–2.º ano, e sem ela o critério vira uma regra arbitrária que um avaliador pode considerar dispensável.

Nuance. Parte da poda é defensável como redução de densidade (Seção 4.5). A diferença é que em M4 removeu-se **redundância**, e aqui removeu-se **ancoragem**. Recomenda-se rejeitar esta parte do *diff* e manter o texto atual, adotando apenas as substituições terminológicas.

4.9 M8 — "aluno" para "estudante"

Padronização consistente com a regra já vigente no P0a/P0b e reafirmada na regra 2 do novo P5b. Sem efeito técnico. Observação de completude: a substituição foi aplicada em `criterios-experiencia.md` e no critério 1.6, mas não nos demais critérios de `criterios-gate-estrutural.md` (1.3 e 1.4 seguem com "aluno"), o que deixa o vocabulário misto. Se a padronização for para valer, convém completá-la.

4.10 M9 — Material *Entre Redes* como fonte primária de BNCC

O que mudou. P2 e P5 recebem um parágrafo novo: quando o material do *Entre Redes* do bimestre vier anexado, ele é a **fonte primária** de código e habilidade, e a lista oficial passa a funcionar como verificação e *fallback*.

Hipótese inicial e sua refutação. A leitura do P5 sugeria um conflito grave: a esteira tem um gate determinístico (`tools/bncc.py::validar_spec`) que valida os códigos do bloco 11 da *spec* contra `bncc-computacao.json`, transcrição do anexo oficial do MEC [2], e emite *bloqueio* para código inexistente. Autorizar o P5 a copiar códigos de outra fonte pareceria abrir caminho para bloqueio sistemático.

A hipótese foi **testada e refutada**. Extraindo os códigos do material `entre_redes_2ano_1_bimestre.docx` presente na raiz do repositório, encontram-se oito códigos, dos quais apenas dois são de Computação:

Código	Situação na lista oficial	Efeito no gate
EF02CO01	presente (eixo PC, 2.º ano)	validado
EF02CO05	presente (eixo CD, 2.º ano)	validado, conta como letramento digital
EF02HI08, EF12LP11, EF12LP14	fora da lista de Computação	**ignorado** pelo gate
EF15AR01, EF15AR04, EF15LP07	fora da lista de Computação	**ignorado** pelo gate

A razão é que a expressão regular do extrator casa somente códigos `EFnnCOnn` e `EM13COnn`. Uma execução do próprio `validar_spec` sobre uma *spec* sintética contendo EF02CO05, EF12LP11 e EF02HI08 retorna `status: aprovado`, sem achados. **Não há bloqueio**, e o alarme inicial era infundado.

O que permanece como risco. Dois pontos, ambos condicionais e de severidade moderada:

1. **Divergência de ano em código de Computação.** Se algum material do *Entre Redes* atribuir um código `CO` a um ano diferente do anexo oficial, o gate bloqueia com "habilidade de outro ano" — e o *prompt* revisado instrui explicitamente a preferir o material. No material de 2.º ano testado isso não ocorre; em outros bimestres e anos, não foi verificado.
2. **Assimetria de validação.** Os códigos de componentes tradicionais entram na *spec* sem nenhuma verificação determinística — o gate simplesmente não os enxerga. Elevar o material a fonte primária aumenta o volume desses códigos não verificados. Não é uma regressão introduzida pelo PO, mas a mudança amplia a exposição a uma lacuna preexistente.

Recomendação. Aceitar a mudança e acrescentar dois itens ao plano: rodar a extração de códigos sobre **todos** os materiais do *Entre Redes* disponíveis e conferir os `CO` contra o JSON oficial (tarefa de minutos, com o instrumento que já existe); e avaliar estender o extrator para validar também os códigos não-`CO` contra a BNCC geral, fechando a lacuna.

4.11 M10 — O novo agente P5b

O que é. Um agente novo que roda ao lado do P5 e produz, a partir da mesma missão aprovada, um documento em prosa para quem vai avaliar pedagogia e não precisa ler máquina de estados, *schema* Zod ou cardinalidade. Estrutura fixa de dez seções, citação de abertura em primeira pessoa, e uma regra de honestidade explícita: toda ressalva sobrevivente e toda decisão de produto pendente aparecem no documento, mesmo que o gate final tenha saído **VERDE**.

Por que é uma boa adição. É separação de audiência, não duplicação de conteúdo. A *spec* do P5 é um contrato de construção, escrito para Design e Dev; usá-la como documento de avaliação pedagógica obriga o leitor pedagógico a atravessar 12 blocos técnicos para extrair o que lhe interessa. São públicos com necessidades de informação distintas, e o critério de decomposição de Parnas [11] — separar por *decisão de projeto encapsulada*, não por etapa de processo — se aplica também a artefatos documentais: o P5b encapsula "como esta missão se explica para quem avalia pedagogia", que é uma decisão independente de "como esta missão se constrói".

A regra de não esconder ressalvas é o complemento correto de M1: se o veredito ternário torna o risco visível no envelope, o P5b garante que ele também fique visível no documento que a pessoa lê.

O que falta para funcionar. O arquivo é, hoje, um *prompt* órfão:

- não há entrada correspondente em `agents/produto/__init__.py` — nem uma função `get_one_pager_pedagogic` nem registro no catálogo de agentes usado pela interface;
- não está incluído no `SequentialAgent` da etapa de Produto, e portanto nunca executa;
- as chaves de estado que ele interpola existem e estão corretas (`gate_estrutural_status`, `gate_final_status`, `avaliacao_pedagogica_report`, `experiencia_aluno_report`, `gate_final_report`), o que indica que o autor conhecia o contrato — só não fez o *wiring*;
- as referências internas do *prompt* (seções 15 e 18 do *one-pager*, blocos `## Changelog` do P1b, estrutura de 18 seções) foram conferidas e **existem** nos artefatos correspondentes.

Decisão de desenho pendente. O P5b consome `gate_final_report`, portanto roda depois do P4, como o P5. Como são independentes entre si, o lugar natural é um `ParallelAgent` com o P5 — ambos leem os mesmos insumos e nenhum consome a saída do outro. Isso adiciona um agente à etapa sem adicionar latência significativa.

4.12 M11 — `formato-spec.md`: a regressão grave

O achado. Este não é um arquivo revisado: é um arquivo **revertido**. A comparação contra o histórico do *git* mostra que a versão entregue difere da versão do *commit* `4dcdb18` (16/09) em apenas 13 linhas — o *frontmatter* YAML e o parágrafo final sobre os blocos 13 e 14. Ou seja, o PO trabalhou sobre um *snapshot* anterior ao `262f160` (17/09), que sincronizou os contratos da plataforma, e o resultado desfaz aquele trabalho.

O saldo é de **73 linhas removidas contra 8 adicionadas**.

O que se perde. Cinco blocos de contrato, dos quais dois com consequência determinística imediata:

1. **A ficha do manifesto (bloco 0).** O formato atual especifica dez campos em formato fixo, um por linha. O formato revisado substitui isso por prosa ("Slug `exp-NN-<nome>` · nome que o estudante vê · série(s)…"). O validador `tools/rastro.py::validar_manifesto` lê exatamente estes dez campos por expressão regular:

   `id` · `produto` · `series` · `unidade` · `publico` · `viewport` · `permissoes` · `capabilities` · `secao_ds` · `liberacao`

   Cada campo ausente ou fora do formato gera um achado de severidade **bloqueio**. O próprio *prompt* atual avisa: *"um gate determinístico lê estas linhas; campo fora do formato conta como ausente"*. O gate `ManifestoCheck` roda na etapa de Dev, com `estrito` ligado por padrão. **Consequência:** adotando este arquivo, toda *spec* gerada perde a ficha, e toda missão trava no início da etapa de Dev — depois de já ter consumido a etapa de Produto inteira e boa parte da de Design.

2. **O bloco de mídia capturada do estudante.** O formato atual exige, sem exceção, quando a missão usa `microphone`, `camera` ou `display-capture`: onde a mídia é gravada (tabela com `escola_id` e RLS, ou *bucket* com caminho não adivinhável), por quanto tempo fica, quem lê (apenas a professora da escola — nunca rota de gestor, nunca telemetria) e o que acontece quando a família não autoriza. O texto é explícito: *"Voz e imagem de criança são dado pessoal e não são coletados por padrão"*. A revisão remove o bloco inteiro. Trata-se de dado pessoal de criança e adolescente, com tratamento específico na LGPD; a exigência não é burocracia interna, é o registro documental de uma obrigação legal.

3. **O contrato de eventos.** A assinatura `track(verbo, objeto, <resultado?, contexto?>)`, a advertência de que **não existe campo `dados`** (a API descarta chave desconhecida em silêncio) e a regra de que nenhum relatório de missão nasce de `telemetria_eventos`. Remover isto não bloqueia nada — produz **falha silenciosa**, que é pior: a missão é construída, a telemetria é escrita, e o dado simplesmente não aparece.

4. **Os cinco caminhos de dado, RLS e fuso.** A distinção entre caminho analítico e operacional (incluindo o aviso de que missão nova **não herda** `POST /runs/:id/eventos`), a exigência de `escola_id NOT NULL` com `FORCE ROW LEVEL SECURITY` em toda tabela `exp_NN_*`, e a regra de que toda data vem do *handshake*.

5. **Os blocos 13 e 14.** A declaração de que a esteira de Design anexa dois blocos à *spec* (decisões D0 e ambiente simulado) e de que a ausência deles indica que D0 ou D9 não rodaram — ou seja, a instrumentação que permite detectar que uma etapa foi pulada.

Classificação. Na taxonomia de Cemri et al. [1], isto é uma falha de especificação de sistema combinada com omissão de informação entre agentes: o produtor (P5) deixa de emitir campos que o consumidor (gate de Dev) exige, e a detecção ocorre longe da origem. É também o cenário que o próprio `sincronizar_plataforma.py` foi escrito para evitar — a diferença é que aquele *script* sincroniza apenas `references/plataforma/`, de modo que **esta regressão não seria recuperada automaticamente**. Se o arquivo for adotado, o conteúdo perdido só volta por reversão manual.

Recomendação. **Descartar integralmente** o `formato-spec.md` revisado. Se houver mudança desejada dentro dele, reaplicar sobre a versão atual.

4.13 M12 — `ambientes.md`: reversão e erro factual

O achado. Mesmo padrão: a versão entregue está a seis linhas da versão de `59b52d8` (02/09), desfazendo a sincronização de 16/09. Perde-se a identificação das pontes em `@edu/ui` (`ds-unootube`, `ds-whatsapp`), a restrição registrada em ADR-68 (WhatsApp disponível apenas em tema escuro) e a marcação de que o TikTok existe no *design system* mas **sem ponte**, sendo a criação da ponte um pré-requisito declarado.

O erro factual. A versão revisada marca o ambiente WhatsApp como "*existe sem consumidor*". A versão atual registra duas missões que o usam: exp-06 (Rastro do Boato) e exp-07 (O Eco no Grupo). Um agente de Design que leia esta tabela concluirá que não há precedente de uso, quando há dois.

Recomendação. **Descartar**.

4.14 M13 — Remoção do *frontmatter* YAML

Quatro arquivos de referência perdem o bloco `-- name / description / sources --`. Verificação: `prompts/loader.py` não lê *frontmatter* — `resolve_includes` injeta o texto como está —, portanto **não há quebra funcional**, e o efeito colateral é até levemente positivo (menos ruído no *prompt* final).

O que se perde é metadado de proveniência: quem escreve o arquivo, quem o lê e de onde veio o conteúdo. Em `padroes-aprendidos.md`, por exemplo, o *frontmatter* registrava que o arquivo é escrito pelo agente de aprendizado e lido pelo de QA de conformidade — informação que não está em nenhum outro lugar. Impacto baixo; mas, somado a M11 e M12, é mais um indício de que o pacote foi montado fora do fluxo de versionamento do repositório.

5 Efeito líquido sobre a construção da missão

5.1 Quantificação

Tabela 4: Densidade de instrução por agente: atual *versus* revisado

Agente	Tokens (atual)	Tokens (rev.)	Critérios (atual)	Critérios (rev.)
P0a núcleo narrativo	13.074	—	0	—
P0mg micro-gate	1.332	—	0	—
P0b seções de apoio	11.661	—	0	—
P1 gate estrutural	8.986	9.300	23	23
P1b revisor	427	—	0	—
P2 avaliação pedagógica	14.311	14.292	20	20
P3 experiência	5.970	5.892	11	**8**
P4 gate final	1.886	1.952	0	0
P5 *spec*	16.100	**14.728**	0	0
P5b *one-pager* ped.	—	1.228	—	0
Total	**73.747**	—	**54**	**51**

Dois números merecem leitura cuidadosa:

- A redução de **11 para 8 critérios no P3** é o ganho pretendido de M4 e é legítima.
- A redução de **1.372 *tokens* no P5 não** é ganho de eficiência: é exatamente o conteúdo de contrato removido por M11. O *prompt* ficou menor porque perdeu a parte que o gate de Dev exige. Neste caso, "mais enxuto" significa "incompleto".

5.2 Matriz de decisão

Tabela 5: Recomendação por mudança

#	Recomendação	Condição
M1	Integrar	junto com a salvaguarda de contagem (R2)
M2	Integrar	**depois** de corrigir `callbacks.py` e `session_store.py`
M3	Integrar	corrigindo o gatilho de rodada 4 para 3
M4	Integrar	documentando a dependência no lado do P1
M5	Integrar	—
M6	Integrar	preferencialmente reescrito como *checklist* objetiva
M7	**Descartar**	manter o texto atual; aplicar só a troca terminológica
M8	Integrar	completando a padronização nos critérios 1.3 e 1.4
M9	Integrar	após validar os códigos `CO` de todos os materiais
M10	Integrar	exige *wiring* em `agents/produto/__init__.py`

M11	**Descartar integralmente**	reaplicar qualquer mudança desejada sobre a versão atual
M12	**Descartar integralmente**	—
M13	Descartar	restaurar o *frontmatter*

5.3 Como o pacote ajuda a construção

1. **Risco deixa de ser invisível.** Hoje, uma missão com três ressalvas aceitas e uma com nenhuma chegam ao Design com o mesmo rótulo. Depois, não. Isso muda o que a etapa seguinte sabe.
2. **O *loop* deixa de girar em falso.** A regra de parada transforma "bloqueado de novo" em "bloqueado por decisão de produto, e o dono é fulano".
3. **O P3 volta a fazer só o que só ele faz.** Três critérios a menos, dedicados a carga cognitiva e faixa etária.
4. **O critério de reflexão fica testável.** A pergunta "um estudante que não fez a missão consegue responder?" é aplicável sem conhecer a intenção do autor.
5. **O time pedagógico ganha um documento próprio.** O P5b elimina a necessidade de ler a *spec* técnica para avaliar pedagogia.

5.4 Como o pacote dificulta

1. **A *spec* deixa de satisfazer o gate de manifesto** (M11) — bloqueio determinístico no início da etapa de Dev, para todas as missões, depois de todo o custo de Produto e Design.
2. **O envelope deixa de ser decidido pelo caminho estruturado** (M2) — a decisão passa a depender de heurística sobre texto, com caminho concreto de falso bloqueio.
3. **Obrigações de dado pessoal de criança saem do contrato** (M11) — sem bloqueio, o que significa que a ausência não é detectada.
4. **Fatos de plataforma regridem** (M12) — incluindo uma afirmação factualmente errada sobre ambientes sem consumidor.
5. **A rubrica de experiência perde âncoras** (M7) — mais variância entre execuções.

6 Granularidade da esteira: nove agentes ou cinco?

6.1 A pergunta

A hipótese de trabalho é que "mais agentes especialistas é melhor do que um que agrega muita coisa". A conclusão desta análise é que **a hipótese está certa para esta esteira, mas o motivo usualmente invocado está errado** — e a diferença importa, porque é o motivo que diz onde a regra deixa de valer.

6.2 O argumento a favor da decomposição

1. Densidade de instrução degrada a aderência, de forma não linear. O *benchmark* IFScale [4] avaliou 20 modelos de fronteira de sete provedores em densidades de 10 a 500 instruções simultâneas. Mesmo os melhores modelos atingem apenas **68% de acurácia** na densidade máxima, com três padrões distintos de degradação e viés sistemático em favor das instruções apresentadas mais cedo. Liu et al. [8] documentam o efeito correlato em contexto longo: o desempenho é mais alto quando a informação relevante está no início ou no fim, e cai significativamente quando ela está no meio — inclusive em modelos projetados para contexto longo.

A Tabela 4 mostra que a etapa de Produto carrega hoje ≈73.700 *tokens* de instrução e 54 critérios tabelados. Uma consolidação em cinco agentes produziria unidades como [P0a+P0mg+P0b] com ≈26.100 *tokens* e [P2+P3] com ≈20.300 *tokens* e **31 critérios em uma única chamada**. É precisamente a faixa em que a degradação é documentada.

2. O projeto já rodou este experimento — e o resultado está escrito no próprio código. O *prompt* do P3 abre justificando a própria existência: *"Esta avaliação existe porque a rubrica original errava sistematicamente aqui: os critérios de faixa etária competiam por atenção com ∼30 critérios de outra natureza. Aqui eles têm o espaço todo."* Este é um experimento natural já realizado no domínio real, com o modelo real e a rubrica real: a versão agregada falhava de forma sistemática e reprodutível; a versão decomposta corrigiu a falha. Nenhuma evidência externa é mais relevante que essa.

3. Cada agente tem critério de aceitação próprio e verificável. Este é o teste que realmente decide a questão. O critério de decomposição de Parnas [11] — o artigo que estabeleceu o conceito — é explícito em que **não se decompõe por etapa cronológica do processo**, e sim por decisão de projeto encapsulada: cada módulo esconde algo que pode mudar sem afetar os demais. Aplicado aqui: o P1 encapsula "a missão tem estrutura construível?"; o P2, "a missão cumpre currículo e guardrails?"; o P3, "um estudante desta idade consegue viver isto?". São perguntas que podem ser respondidas, e erradas, independentemente. Cada uma emite um envelope JSON verificável, e cada envelope é lido por um `EnvelopeGateAgent` que decide seguir, repetir ou parar.

4. Localização de falha e custo de correção. Com nove envelopes, a pergunta "onde isto quebrou?" tem resposta. Com cinco agregadores, a falha fica dentro de um agente e o *loop* de correção perde alvo. Mais importante: os dois *loops* existentes (P0a↔P0mg e P1↔P1b) só são economicamente viáveis porque a unidade regenerada é pequena. Reexecutar um agregador de 26 mil *tokens* para corrigir um *spoiler* na seção 4 é desperdício por construção.

5. Separar quem julga de quem corrige reduz autopreferência. O P1b (427 *tokens*) parece pequeno demais para ser um agente. Mas fundi-lo ao P1 faria o mesmo contexto avaliar a própria correção. Zheng et al. [17] documentam viés de autopreferência em avaliadores baseados em modelos de linguagem; manter contextos separados entre o avaliador e o corretor é uma mitigação barata. O tamanho do *prompt* não é o critério — o papel é.

6.3 O argumento contrário, e por que ele não se aplica aqui

Honestidade intelectual exige registrar que a literatura recente é **cética** quanto a sistemas multiagente, e por boas razões.

Ganhos frequentemente explicados por computação extra. Tran e Kiela [15] apresentam um argumento informacional, baseado na desigualdade do processamento de dados, de que sob orçamento fixo de *tokens* de raciocínio e com utilização perfeita de contexto, o sistema de agente único é mais eficiente em informação — e confirmam empiricamente, em três famílias de modelos, que o agente único iguala ou supera arquiteturas multiagente em raciocínio *multi-hop* quando a computação é normalizada. A conclusão dos autores é direta: muitas vantagens relatadas de sistemas multiagente se explicam melhor por computação não contabilizada do que por benefício arquitetural.

Por que não se aplica. O próprio artigo enuncia a condição de contorno: sistemas multiagente *"tornam-se competitivos quando a utilização efetiva de contexto de um agente único está degradada"*. Com 73.700 *tokens* de instrução e 54 critérios, a esteira está claramente nesse regime — é o caso previsto pelo paper, não o contraexemplo. Além disso, a tarefa medida ali é raciocínio *multi-hop* com resposta única e verificável; esta esteira produz **artefatos intermediários auditáveis** com critérios de aceitação heterogêneos, onde o valor da decomposição não está na acurácia final e sim na atribuição de responsabilidade.

Mais fronteiras significam mais modos de falha. Cemri et al. [1] catalogam 14 modos de falha em três categorias, e duas delas — especificação de sistema e desalinhamento entre agentes — são consequência direta de ter fronteiras. Este relatório é, ele próprio, evidência: M2 e M11 são exatamente falhas de contrato de interface, do tipo que não existiria em um agente monolítico.

A leitura correta não é "decomponha menos"; é "decomponha e **trate as interfaces como contratos de primeira classe**", no sentido de Meyer [9]. O custo da decomposição é ter de manter contratos; quem decompõe sem manter contrato paga os dois preços.

6.4 O critério de decisão

Da análise resulta uma regra operacional em três testes. **Decomponha** quando os três forem verdadeiros:

1. **Aceitação independente** — o sub-resultado tem critério de aceitação verificável sem consultar os demais agentes (na prática: emite um envelope próprio que um gate consegue ler);
2. **Correção localizável** — quando falha, existe uma ação de correção que não obriga a refazer o restante;
3. **Densidade** — a fusão colocaria o agente resultante acima da faixa segura de instruções simultâneas.

E **não decomponha** quando: a divisão for apenas cronológica ("primeiro faz X, depois Y" sem que X tenha aceitação própria); os dois avaliadores aplicarem o **mesmo teste** (é o caso de M4 — ali a resposta certa foi remover, não separar melhor); ou a fronteira criar um contrato que ninguém vai manter.

6.5 Conclusão e recomendação

Nove é melhor que cinco nesta esteira — não porque especialização seja um bem em si, mas porque os nove passam nos três testes, enquanto as fusões plausíveis para chegar a cinco reprovam no primeiro e no terceiro.

Vale ir além da pergunta feita: **a direção correta é aumentar a granularidade, não reduzi-la**. Dois movimentos concretos:

- **O P5b já move a esteira para dez agentes**, e por motivo correto (separação de audiência). O PO está aplicando a mesma lógica que este relatório defende.
- **O P2 é o próximo candidato natural a divisão.** Com 14.311 *tokens* e 20 critérios, é hoje o maior avaliador, e mistura três lentes distintas: conformidade curricular (BNCC, letramento digital), mediação docente (gatilhos, painel, roteiro de perguntas) e guardrails de proteção do estudante (exposição, comparação nominal, IA em tempo real). São três perguntas com critérios de aceitação distintos e correção localizável — exatamente o perfil que justificou a criação do P3. Separar currículo de guardrails é o movimento com melhor relação custo-benefício disponível hoje na etapa.

Contrapartida obrigatória: cada novo agente adiciona uma interface, e interface sem contrato mantido é dívida. Antes de dividir o P2, vale fechar a dívida que M2 e M11 expõem — um teste automatizado que valide que todo valor de `status` emitido pelos *prompts* pertence ao vocabulário que `inferir_status` reconhece custa poucas linhas e teria capturado M2 antes da entrega.

7 Recomendações

R1 — Reverter M11 e M12 antes de qualquer outra coisa.
Descartar `formato-spec.md` e `ambientes.md` do pacote. São reversões de contrato, não revisões. Se houver mudança desejada neles, reaplicá-la sobre a versão em `main`.

R2 — Corrigir o vocabulário de *status* no código.
Acrescentar `aprovado_com_ressalvas` às listas de `agents/callbacks.py::inferir_status` e de `pipeline/session_store.py::formatar_status_label`. Duas linhas, e destrava M1.

R3 — Fechar a lacuna de contrato com um teste.
Teste automatizado que extraia todos os valores de `status` declarados nos *prompts* e afirme que cada um é reconhecido por `inferir_status`. Isso transforma a classe inteira de falha em erro de CI.

R4 — Ajustar o gatilho da regra de parada.
Trocar "rodada 4" por "rodada final" ou parametrizar pelo valor de `GATE_ESTRUTURAL_MAX_RODADAS` via `load_prompt(**kwargs)`, eliminando a possibilidade de divergência futura.

R5 — Fazer o *wiring* do P5b.
Registrar o agente em `agents/produto/__init__.py`, adicioná-lo ao catálogo e colocá-lo em `ParallelAgent` com o P5.

R6 — Instrumentar a contagem de ressalvas.
Persistir `ressalvas` por missão e acompanhar a série. Se a média subir sem queda correspondente em bloqueios, é deriva de leniência — o risco previsto na Seção 4.2.

R7 — Medir, em vez de argumentar, o efeito sobre a qualidade final.
Este relatório avalia contratos e desenho de instrumento; não mede qualidade de missão. O teste adequado é executar $n \geq 6$ missões do mesmo bimestre nos dois braços (prompts atuais *versus* revisados com R1–R4 aplicados) e submeter os *one-pagers* a avaliação cega por dois especialistas pedagógicos, medindo concordância. Com a infraestrutura de execução por etapa isolada que já existe no projeto, o custo é baixo.

R8 — Estabelecer higiene de revisão de *prompt*.
A causa raiz de M11, M12 e M13 não é julgamento ruim — é ter editado uma cópia desatualizada fora do *git*. Revisões de *prompt* devem sair de *branch* do repositório e chegar como *pull request*, onde o *diff* contra `main` torna uma reversão de 73 linhas imediatamente visível.

8 Conclusão

A desconfiança em relação ao pacote não se confirma no plano que importava verificar. O julgamento pedagógico e de desenho de avaliação do *Product Owner* é sólido: das treze mudanças, oito são melhorias, e duas delas — a desduplicação de critérios correlacionados e a operacionalização do critério de reflexão — são o tipo de correção que costuma passar despercebida por quem não conhece o instrumento a fundo.

O que o pacote não tem é disciplina de integração. As duas regressões graves não são erros de conteúdo: são o resultado mecânico de editar uma cópia desatualizada fora do controle de versão. É um problema de processo com solução de processo (R8), não um problema de competência.

Quanto à arquitetura: a intuição de que mais agentes especialistas é melhor está correta para esta esteira, e o próprio repositório já contém a evidência — a nota no *prompt* do P3 registra que a rubrica agregada falhava sistematicamente e a decomposição resolveu. Mas a regra não é "mais é melhor"; é "decomponha onde há critério de aceitação próprio, correção localizável e densidade

excessiva, e trate cada fronteira criada como um contrato que alguém precisa manter". Os dois defeitos de integração descritos neste relatório são a fatura dessa última cláusula.

Referências

[1] CEMRI, M.; PAN, M. Z.; YANG, S. et al. *Why Do Multi-Agent LLM Systems Fail?* Advances in Neural Information Processing Systems 38 (NeurIPS 2025), Datasets and Benchmarks Track, 2025. arXiv:2503.13657. Disponível em: `https://arxiv.org/abs/2503.13657`.

[2] BRASIL. Conselho Nacional de Educação. Câmara de Educação Básica. *Resolução CNE/-CEB n.º 1, de 4 de outubro de 2022*: estabelece normas sobre Computação na Educação Básica — Complemento à Base Nacional Comum Curricular (BNCC). Brasília: MEC, 2022. Disponível em: `https://www.gov.br/mec/pt-br/cne/pdf/bncc-ensino-medio/resolucao-cne-ceb-no-1-de-4-de-outubro-de-2022`.

[3] COWAN, N. The magical number 4 in short-term memory: a reconsideration of mental storage capacity. *Behavioral and Brain Sciences*, v. 24, n. 1, p. 87–114, 2001.

[4] JAROSLAWICZ, D.; WHITING, B.; SHAH, P.; MAAMARI, K. *How Many Instructions Can LLMs Follow at Once?* arXiv:2507.11538, jul. 2025. Disponível em: `https://arxiv.org/abs/2507.11538`.

[5] JONSSON, A.; SVINGBY, G. The use of scoring rubrics: reliability, validity and educational consequences. *Educational Research Review*, v. 2, n. 2, p. 130–144, 2007. DOI: 10.1016/j.edurev.2007.05.002.

[6] KHOT, T.; TRIVEDI, H.; FINLAYSON, M. et al. *Decomposed Prompting: A Modular Approach for Solving Complex Tasks*. International Conference on Learning Representations (ICLR), 2023. arXiv:2210.02406.

[7] KNIGHT, J. C.; LEVESON, N. G. An experimental evaluation of the assumption of independence in multiversion programming. *IEEE Transactions on Software Engineering*, v. SE-12, n. 1, p. 96–109, 1986.

[8] LIU, N. F.; LIN, K.; HEWITT, J. et al. Lost in the middle: how language models use long contexts. *Transactions of the Association for Computational Linguistics*, v. 12, p. 157–173, 2024. DOI: 10.1162/tacl_a_00638.

[9] MEYER, B. Applying "design by contract". *IEEE Computer*, v. 25, n. 10, p. 40–51, 1992.

[10] MILLER, G. A. The magical number seven, plus or minus two: some limits on our capacity for processing information. *Psychological Review*, v. 63, n. 2, p. 81–97, 1956.

[11] PARNAS, D. L. On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, v. 15, n. 12, p. 1053–1058, 1972.

[12] PRESTON, C. C.; COLMAN, A. M. Optimal number of response categories in rating scales: reliability, validity, discriminating power, and respondent preferences. *Acta Psychologica*, v. 104, n. 1, p. 1–15, 2000.

[13] SAAL, F. E.; DOWNEY, R. G.; LAHEY, M. A. Rating the ratings: assessing the psychometric quality of rating data. *Psychological Bulletin*, v. 88, n. 2, p. 413–428, 1980.

[14] SWELLER, J. Cognitive load during problem solving: effects on learning. *Cognitive Science*, v. 12, n. 2, p. 257–285, 1988.

[15] TRAN, D.; KIELA, D. *Single-Agent LLMs Outperform Multi-Agent Systems on Multi-Hop Reasoning Under Equal Thinking Token Budgets*. arXiv:2604.02460, abr. 2026. Disponível em: `https://arxiv.org/abs/2604.02460`.

[16] WU, T.; TERRY, M.; CAI, C. J. AI Chains: transparent and controllable human-AI interaction by chaining large language model prompts. In: *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems (CHI '22)*. Nova Orleans: ACM, 2022. DOI: 10.1145/3491102.3517582.

[17] ZHENG, L.; CHIANG, W.-L.; SHENG, Y. et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. Advances in Neural Information Processing Systems 36 (NeurIPS 2023), 2023. arXiv:2306.05685.

A Evidências verificadas

Tabela 6: Afirmações deste relatório e como foram verificadas

Afirmação	**Verificação**
`formato-spec.md` revisado é a versão de `4dcdb18`	`git show 4dcdb18:prompts/references/formato-spec.md` difere do arquivo entregue em 13 linhas (*frontmatter* + parágrafo dos blocos 13/14).
`ambientes.md` revisado é a versão de `59b52d8`	Mesmo procedimento; 6 linhas de diferença.
Os dez campos do manifesto são lidos por validador determinístico	`grep '_campo(texto, "' tools/rastro.py` retorna `capabilities, id, liberacao, permissoes, produto, publico, secao_ds, series, unidade, viewport`; cada ausência chama `falta()`, severidade `bloqueio`.
`aprovado_com_ressalvas` não é reconhecido	Leitura de `agents/callbacks.py:55-73` e `pipeline/session_store.py:53-68`.
Tokens de bloqueio são testados antes dos de aprovação	`agents/callbacks.py:69-72`.
O teto do *loop* é 3	`config.py:97` (`GATE_ESTRUTURAL_MAX_RODADAS`, padrão 3) e `.env`; usado como `max_iterations` em `agents/produto/__init__.py:182`.
3.1≡1.3, 3.2≡1.4, 3.3≡7.2	Comparação literal dos testes práticos em `criterios-gate-estrutural.md:9,10,54` e `criterios-experiencia.md` (versão atual).
O gate BNCC ignora códigos não-`CO`	`tools/bncc.py:47` (expressão regular) e execução de `validar_spec` sobre *spec* sintética com EF02CO05 + EF12LP11 + EF02HI08 → `status: aprovado`.
O material de 2.º ano usa EF02CO01 e EF02CO05	Extração dos códigos de `entre_redes_2ano_1_bimestre.docx` e conferência contra `bncc-computacao.json` (130 habilidades).
P5b não está registrado	Ausência do arquivo em `load_prompt(...)` e no catálogo de agentes de `agents/produto/__init__.py`.

Densidade por agente (Tabela 4)	`prompts.loader.load_prompt` com resolução de *includes*; contagem de critérios por correspondência de linhas de tabela `|n.n|`; *tokens* estimados em `chars/4`.